

LAPORAN
SIMULASI ROUND ROUND ROBIN DAN LEAST CONNECTION



DISUSUN OLEH :
SARLINDA SOPALATU
105841101423
RPL_A

PRODI INFORMATIKA
FAKULTAS TEKNIK
UNIVERISTAS MUHAMMADIYAH MAKASSAR
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Ketersediaan infrastruktur peladen (server) yang andal merupakan kebutuhan absolut bagi sistem informasi akademik, khususnya pada implementasi portal ujian online serentak seperti pada sistem MIS Hidayatullah KM 12. Ketika ratusan siswa melakukan proses otentikasi dan mengakses butir soal secara bersamaan dalam jendela waktu yang sempit, peladen tunggal rentan mengalami kondisi beban berlebih (overload) yang berujung pada kelumpuhan layanan (downtime). Untuk menjamin stabilitas aplikasi, diperlukan arsitektur jaringan yang mengimplementasikan teknologi Load Balancing. Pemodelan infrastruktur ini kini dapat disimulasikan secara efisien menggunakan teknologi containerization seperti Docker, yang dipadukan dengan Nginx sebagai agen reverse proxy, guna membandingkan efektivitas distribusi beban menggunakan algoritma statis (Round Robin) dan dinamis (Least Connection).

2.1 Rumusan Masalah

- a. Bagaimana merancang dan mengorkestrasi simulasi infrastruktur *load balancing* menggunakan Docker dan Nginx?
- b. Bagaimana perbandingan perilaku Nginx saat mendistribusikan lalu lintas menggunakan algoritma Round Robin dibandingkan Least Connection?

3.1 Tujuan

Merancang simulasi kluster peladen berbasis kontainer untuk menguji, memantau, dan menganalisis secara langsung proses distribusi beban lalu lintas HTTP berdasarkan metode statis dan adaptif.

BAB II

PEMBAHASAN

2. 1Arsitektur Lingkungan Simulasi

Topologi yang dibangun pada simulasi ini terdiri dari empat entitas kontainer yang berjalan secara virtual dalam satu mesin lokal. Nginx bertindak sebagai gerbang utama (*Load Balancer*) yang mengekspos *port* 8080, sementara di belakangnya terdapat tiga kontainer Apache/PHP (*web_a*, *web_b*, *web_c*) yang bertindak sebagai *backend server* untuk mengolah permintaan *client*.

2. 2 Implementasi dan Penjelasan Konfigurasi Kode

a. Orkestrasi Infrastruktur (File docker-compose.yml)

Untuk membangun keempat entitas peladen secara bersamaan tanpa konfigurasi manual, digunakan berkas orkestrasi YAML.

```
lb_nginx:
  image: nginx:alpine
  ports:
    - "8080:80"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
  depends_on:
    - web_a
    - web_b
    - web_c
```

Blok kode di atas menginstruksikan *Docker Engine* untuk mengunduh sistem operasi mini *nginx:alpine*. Direktif *ports*: "8080:80" berfungsi memetakan akses lalu lintas dari *browser* ke dalam kontainer, sedangkan *depends_on* memastikan bahwa Nginx hanya akan menyala jika ketiga *backend server* (*web_a*, *web_b*, *web_c*) sudah berhasil dihidupkan (*booting*) terlebih dahulu.

b. Konfigurasi Mesin Distribusi (File nginx.conf)

File ini merupakan inti pengambil keputusan (*decision maker*) yang menentukan ke mana permintaan dari klien akan dilempar.

```
upstream backend_cluster {
    # least_conn;
    server web_a:80;
    server web_b:80;
    server web_c:80;
}
```

Blok *upstream* berfungsi mendefinisikan sebuah grup klaster peladen. Secara bawaan (seperti kode di atas), Nginx akan mengaktifkan algoritma **Round Robin**,

di mana permintaan dibagi secara berurutan. Untuk mengubahnya menjadi sistem dinamis yang mengecek beban *real-time*, baris perintah `least_conn`; (yang dikomentari) harus diaktifkan. Algoritma ini akan memaksa Nginx mencari peladen dengan koneksi aktif paling sedikit sebelum melempar permintaan klien.

c. **Skrip Identifikasi Peladen (File `index.php`)**

Berkas PHP ini ditanamkan pada setiap kontainer *backend* untuk mengidentifikasi keberhasilan distribusi lalu lintas jaringan.

```
<?php
```

```
$server_id = gethostname();
```

```
echo "<h1>Kamu sedang dilayani oleh Server: " . $server_id . "</h1>";
```

```
?>
```

digunakan secara dinamis untuk menangkap ID mesin kontainer yang sedang mengeksekusi berkas tersebut. Ini membuktikan kepada klien secara visual bahwa *request* yang dilakukan berulang kali (melalui *refresh*) dilayani oleh mesin fisik/virtual yang berbeda-beda.

2.3 Pengujian Simulasi

Berdasarkan eksekusi perintah `docker-compose up -d` di terminal CLI, observasi menunjukkan perbedaan signifikan:

- a. Pada mode Round Robin: Terjadi perpindahan ID container secara rotasi kaku (A -> B -> C -> A), tanpa mempedulikan apakah pengguna menahan sesi terlalu lama atau tidak.
- b. Pada mode Least Connection: Nginx secara adaptif menahan lalu lintas dari peladen yang merespons lambat, dan memprioritaskan peladen lain yang sudah selesai merender halaman

BAB III

PENUTUP

3.1 Kesimpulan

Dari implementasi berbasis Docker yang telah dilakukan, dapat disimpulkan bahwa, Penggunaan orkestrasi docker-compose terbukti sangat efisien untuk merancang dan menduplikasi infrastruktur jaringan lokal (1 Nginx dan 3 *Backend*) secara instan, konsisten, dan terisolasi dari sistem operasi utama. Nginx bertindak sebagai *Load Balancer* yang tangguh. Algoritma Round Robin sangat ringan dan mudah diterapkan, namun rentan menyebabkan penumpukan antrean pada aplikasi dinamis. Sebaliknya, modifikasi menggunakan direktif `least_conn` memungkinkan sistem beradaptasi terhadap kepadatan sesi pengguna ujian, menjaga peladen agar tidak mengalami kelebihan beban.

3.2 Saran

Untuk pengembangan selanjutnya, disarankan untuk mengintegrasikan alat pemantauan metrik *real-time* berbasis *dashboard* (seperti Grafana atau Prometheus) ke dalam berkas `docker-compose.yml`. Hal ini bertujuan untuk mengukur latensi waktu pengerjaan dan fluktuasi CPU di tiap kontainer secara spesifik, serta melakukan pengujian tekanan batas maksimal (*stress testing*) terhadap Nginx. Dengan format di atas, laporanmu terlihat sangat teknis dan profesional. Ingat, letakkan keseluruhan *full source code* HTML/PHP/YML/Conf yang panjang ke halaman **Lampiran**. Apakah ada bagian dari penjelasan kode (Bab 2.2) yang ingin ditambahkan detailnya?

3.3 Output

a. Terminal

```
PS C:\Users\ASUS\Sarlindasopalatu> docker-compose up -d
empty compose file
PS C:\Users\ASUS\Sarlindasopalatu> docker-compose up -d
time="2026-06-08T23:11:47+08:00" level=warning msg="C:\\Users\\ASUS\\Sa
rlindasopalatu\\docker-compose.yml: the attribute `version` is obsolete
, it will be ignored, please remove it to avoid potential confusion"
[+] up 5/5
✓ Network sarlindasopalatu_default          Created      0.7s
✓ Container sarlindasopalatu-web_a-1        Started      60.0s
✓ Container sarlindasopalatu-web_b-1        Started      60.4s
✓ Container sarlindasopalatu-web_c-1        Started      60.4s
✓ Container sarlindasopalatu-lb_nginx-1     Started      68.3s
PS C:\Users\ASUS\Sarlindasopalatu>
```

b. Browser

